A Comparative Study of CNN and HOG Models for Caltech-101 Image Classification

Chenxi Ge
Harvard University
steven_ge@fas.harvard.edu
https://github.com/chenxige0509/Caltech101-MiniProject

March 2, 2026

Abstract

We address image classification on the Caltech-101 dataset using both transfer learning with deep networks and classical pipelines based on hand-crafted features. We compare three pretrained architectures (ResNet-18, EfficientNet-B0, and Vision Transformer (ViT-B/16)) with frozen backbones and a trainable head, alongside HOG features with SVM and logistic regression. We ablate input resolution, data augmentation, and optimizer choice, and report accuracy, top-5 accuracy, and per-class precision, recall, and F1. ViT-B/16 achieves the best test accuracy (95.93%); ResNet-18 and EfficientNet-B0 reach 92.55% and 94.62% at 224×224; classical HOG+LR reaches 66.05% at 64×64.

1 Introduction

We consider image classification on Caltech-101 [1] with 101 object categories. Understanding how design choices (resolution, augmentation, optimizer) and feature type (hand-crafted vs. learned) affect performance is useful for low-resource or interpretability-sensitive settings. We use (i) transfer learning from ImageNet-pretrained [6] deep networks (ResNet-18 [2], EfficientNet-B0 [3], ViT-B/16 [4]) with only the classification head trained, and (ii) classical pipelines using HOG features [5] with SVM and logistic regression. The main comparisons are summarized in Table 1. We describe data processing and model architectures in Section 2, experimental setup and results in Section 3 and Section 4, and discuss these four dimensions in Section 5; Section 6 concludes.

Table 1: Comparison dimensions and options considered in this study.

Dimension	Options
Image size	64×64, 128×128, 224×224 (where applicable; ViT fixed at 224)
Data augmentation	With vs. without (CNNs only)
Feature extractor	HOG (hand-crafted) vs. CNN (ResNet-18, EfficientNet-B0, ViT-B/16)
Optimizer	SGD vs. Adam (and AdamW) for CNN; N/A for HOG

2 Methods

This section covers data processing (splits, preprocessing, augmentation) and the architectures of all models used.

2.1 Data Processing

Dataset and splits. We use the Caltech-101 dataset [1]: 101 object categories, 8,677 images after excluding the `BACKGROUND_Google` clutter class. Class sizes are imbalanced (roughly 31–800 images per category). With a fixed random seed we perform a stratified 70/15/15 split into train (6,073), validation (1,302), and test (1,302) sets; the same split is used for all models and ablations.

Preprocessing for deep learning. Images are loaded in RGB and resized to a target resolution (64, 128, or 224 pixels per side, depending on the experiment). For training we apply random resized crop (scale 0.8–1.0), random horizontal flip, and color jitter (brightness, contrast, saturation), then convert to tensor and normalize with ImageNet [6] mean and standard deviation. For validation and test we only resize to the same resolution, then normalize. One ablation disables augmentation (resize and normalize only).

Preprocessing for classical baselines. For HOG-based pipelines we load images, convert to grayscale, and resize to the target size (64 or 128). We then compute Histogram of Oriented Gradients (HOG) [5] features: each image is described by local gradient orientation histograms over overlapping cells and blocks (orientations=9, pixels per cell 8×8, cells per block 2×2, L2-Hys block normalization). HOG summarizes edge and contour structure in a fixed-length vector and does not capture texture or high-level semantics. The resulting feature vectors are standardized with `StandardScaler` (fit on train, transform on train/val/test). We report test accuracy, top-5 accuracy, and F1 (macro and weighted) for all HOG baselines in the results.

2.2 Model Architectures

ResNet-18. ResNet-18 [2] is a 18-layer residual network: initial conv layer, four stages of residual blocks (2+2+2+2 blocks) with 64, 128, 256, 512 channels, global average pooling, and a fully connected layer. We load ImageNet-pretrained weights from torchvision, freeze all parameters except the final layer, and replace the final layer with a linear layer mapping to 101 classes. Input size can be 64, 128, or 224 (spatially); the backbone adapts to the input size through the global pooling.

EfficientNet-B0. EfficientNet-B0 [3] uses compound scaling (depth, width, resolution) and mobile inverted bottleneck (MBConv) blocks with squeeze-and-excitation. We use the torchvision implementation with ImageNet pretrained weights, freeze the entire backbone, and replace the classifier (a dropout and linear layer) with a new dropout and linear layer for 101 classes. Input resolution is varied in ablations (64, 128, 224).

Vision Transformer (ViT-B/16). ViT-B/16 [4] splits the image into 16×16 patches, embeds them linearly, adds a learnable [CLS] token and positional embeddings, and processes the sequence with a Transformer encoder (12 layers, hidden size 768, 12 heads). The classification head is a linear layer on the [CLS] token. We use the torchvision ViT-B/16 with ImageNet weights; input is fixed at 224×224. We do not ablate image size for ViT: the pretrained positional embeddings are defined for a fixed 14×14 patch grid (224/16), so 64×64 or 128×128 would require interpolating them to a different grid size (e.g., 4×4 or 8×8), and at 64×64 the model would see only 16 patches versus 196 at 224, which would poorly match the pretrained distribution. We therefore keep ViT at its native 224×224 for a fair transfer-learning comparison. We freeze all parameters except the final head.

HOG + SVM. We extract a fixed-length HOG descriptor per image (as in Section 2.1). A linear SVM (`LinearSVC`) [11] is trained on the standardized features in a one-vs-rest scheme. For probability estimates and top-k accuracy we wrap the SVM with `CalibratedClassifierCV` (sigmoid method, 2-fold cross-validation).

HOG + Logistic Regression. Using the same HOG features and scaling, we train a multinomial logistic regression (L2 regularization, LBFGS solver) for 101-way classification. The single multinomial model directly learns decision boundaries among all 101 classes. No calibration step is needed for probabilities.

3 Experiments

3.1 Training Setup

Table 2 summarizes the training configuration. For CNNs we minimize cross-entropy loss with mini-batch SGD; we use Adam (default), AdamW, or SGD with momentum as ablations. Learning rate and weight decay are fixed per run; we use a cosine annealing learning-rate schedule over the training epochs. For classical models we fit SVM and logistic regression on the full (standardized) training set after HOG extraction.

Table 2: Training setup: CNN and classical pipelines.

Setting	CNN (ResNet-18, EfficientNet-B0, ViT-B/16)	Classical (HOG+SVM, HOG+LR)
Loss	Cross-entropy	N/A (SVM/LR fit)
Optimizer	Adam (default), AdamW, or SGD (momentum)	N/A
Learning rate	Fixed per run; cosine annealing schedule	N/A
Epochs	25	N/A (single fit on full train set)
Batch size	32	N/A
DataLoader `num_workers`	12	N/A (HOG extracted offline)
Weight decay	Fixed per run	L2 (SVM/LR)
Data	Same stratified train/val/test splits	Same splits; HOG on same images

3.2 Ablation Design

We conduct comparisons along four dimensions:

- **Image size**: compare 64×64 vs. 128×128 (and 224×224 for ResNet and EfficientNet where applicable). ViT-B/16 is not ablated over resolution because its pretrained positional embeddings assume a 224×224 input (see Section 2.2).
- **Data augmentation**: train with vs. without augmentation (CNNs only).
- **Feature extractor choice**: HOG (hand-crafted) vs. CNN features (ResNet-18, EfficientNet-B0, ViT-B/16 transfer learning).
- **Optimizer**: SGD vs. Adam for CNN; we also include AdamW for completeness.

All CNN ablations (image size, augmentation, optimizer) use the same evaluation protocol; classical HOG+SVM and HOG+LR runs provide the feature-extractor comparison.

3.3 Evaluation

We report overall accuracy and top-5 accuracy on the test set, and per-class precision, recall, and F1 with macro and weighted averages. Confusion matrices are produced for all baselines (including HOG+SVM and HOG+LR at 64 and 128); per-class accuracy and F1 bar plots are produced for the CNN baselines only (see Section 4). Reproducibility is ensured by a fixed random seed and, for PyTorch, deterministic worker initialization when using multiple DataLoader workers. All results are from a single train/val/test split; we report point estimates without confidence intervals.

4 Results

4.1 Overall Performance and Best in Model Families

Among the CNN models, **ViT-B/16** achieves the best overall performance (95.93% test, 99.23% top-5 with 224×224, no augmentation, Adam). Within each family we report the best setting in Table 3: ResNet-18 reaches 92.55%, EfficientNet-B0 94.62%, and the classical baselines HOG+SVM and HOG+LR (at 64×64 and 128×128) reach 54.61% and 66.05% respectively. HOG+LR attains higher accuracy than HOG+SVM at both resolutions. Table 4 gives the full ablation comparison across all model architectures (image size, augmentation, optimizer).

Table 3: Best test and top-5 accuracy in each model family. Best in each family in bold; best in each column overall marked with *.

Model family	Best setting	Test acc. (%)	Top-5 acc. (%)
ResNet-18	img224, aug, Adam	**92.55**	**98.54**
EfficientNet-B0	img224, aug, Adam	**94.62**	**98.92**
ViT-B/16	224, no aug / aug, Adam	**95.93***	**99.23***
HOG+SVM	img64	**54.61**	**69.97**
HOG+LR	img64	**66.05**	**82.49**

4.2 Full Ablation Comparison (All Model Architectures)

Table 4 lists all ablation runs: HOG (4 runs), ResNet-18 and EfficientNet-B0 (18 runs each: 64/128/224 × augment × Adam/AdamW/SGD), and ViT-B/16 (4 runs at 224×224). Columns are model, input resolution, augmentation, optimizer, test accuracy, top-5 accuracy, and F1 macro/weighted. Best in each family is in bold; best in each column overall is marked with *.

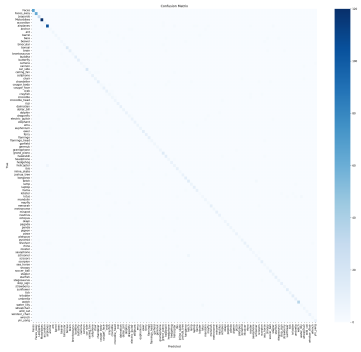
Table 4: Full ablation comparison: image size (64/128/224), data augmentation, feature extractor (HOG vs. CNN), and optimizer. Best in each family in bold; best in each column overall marked with *.

Model	Img size	Aug	Optimizer	Acc. (%)	Top-5 (%)	F1 (Macro)	F1 (Wtd)
HOG+SVM	64×64	False	N/A	54.61	69.97	0.345	0.533
HOG+SVM	128×128	False	N/A	49.92	59.52	0.274	0.464
HOG+LR	64×64	False	N/A	**66.05**	**82.49**	**0.489**	**0.653**
HOG+LR	128×128	False	N/A	65.90	81.49	0.488	0.647
ResNet-18	64×64	True	ADAM	60.60	81.34	0.504	0.590
ResNet-18	64×64	True	AdamW	61.60	81.95	0.523	0.604
ResNet-18	64×64	True	SGD	59.68	80.34	0.485	0.574
ResNet-18	64×64	False	ADAM	66.21	83.72	0.526	0.653
ResNet-18	64×64	False	AdamW	66.21	83.64	0.527	0.653
ResNet-18	64×64	False	SGD	64.44	82.64	0.499	0.628
ResNet-18	128×128	True	ADAM	85.94	96.31	0.811	0.857
ResNet-18	128×128	True	AdamW	85.94	96.24	0.810	0.857
ResNet-18	128×128	True	SGD	83.33	96.16	0.779	0.831
ResNet-18	128×128	False	ADAM	85.79	95.62	0.801	0.856
ResNet-18	128×128	False	AdamW	85.64	95.62	0.799	0.855
ResNet-18	128×128	False	SGD	84.18	96.31	0.777	0.838
ResNet-18	224×224	True	ADAM	**92.55**	**98.54**	**0.889**	**0.924**
ResNet-18	224×224	True	AdamW	92.32	98.62	0.887	0.921
ResNet-18	224×224	True	SGD	91.40	98.16	0.867	0.910
ResNet-18	224×224	False	ADAM	92.40	98.46	0.888	0.921
ResNet-18	224×224	False	AdamW	92.40	98.46	0.888	0.921
ResNet-18	224×224	False	SGD	91.24	98.23	0.865	0.909
EfficientNet-B0	64×64	True	ADAM	57.07	77.73	0.445	0.555
EfficientNet-B0	64×64	True	AdamW	57.07	76.57	0.443	0.556
EfficientNet-B0	64×64	True	SGD	54.07	75.65	0.373	0.493
EfficientNet-B0	64×64	False	ADAM	57.68	77.73	0.418	0.559
EfficientNet-B0	64×64	False	AdamW	57.68	77.65	0.418	0.559
EfficientNet-B0	64×64	False	SGD	55.84	75.27	0.359	0.510
EfficientNet-B0	128×128	True	ADAM	83.95	95.24	0.775	0.836
EfficientNet-B0	128×128	True	AdamW	85.10	95.47	0.784	0.847
EfficientNet-B0	128×128	True	SGD	80.11	93.39	0.696	0.782
EfficientNet-B0	128×128	False	ADAM	84.64	95.08	0.776	0.841
EfficientNet-B0	128×128	False	AdamW	84.79	95.31	0.779	0.844
EfficientNet-B0	128×128	False	SGD	79.19	93.32	0.684	0.771
EfficientNet-B0	224×224	True	ADAM	**94.62**	**98.92**	**0.925**	**0.945**
EfficientNet-B0	224×224	True	AdamW	94.32	98.69	0.919	0.942
EfficientNet-B0	224×224	True	SGD	89.55	97.70	0.833	0.884
EfficientNet-B0	224×224	False	ADAM	92.86	98.23	0.900	0.927
EfficientNet-B0	224×224	False	AdamW	93.16	98.39	0.904	0.931
EfficientNet-B0	224×224	False	SGD	88.71	97.70	0.819	0.876
ViT-B/16	224×224	False	ADAM	**95.93***	99.23	**0.943***	**0.959***
ViT-B/16	224×224	True	ADAM	95.55	**99.31***	0.939	0.955
ViT-B/16	224×224	True	SGD	93.78	**99.31***	0.913	0.936
ViT-B/16	224×224	True	AdamW	95.55	**99.31***	0.939	0.955

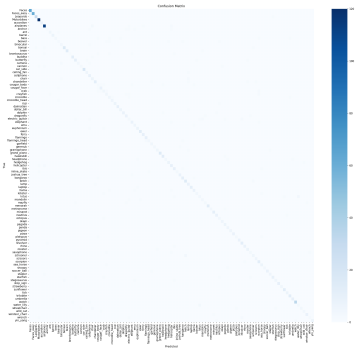
4.3 Baseline Confusion Matrices

The following figure shows the confusion matrix for each baseline. Rows are true labels, columns are predicted labels; diagonal entries are correct predictions. Darker shading indicates higher counts. We include three CNN baselines and four classical baselines (HOG+SVM and HOG+LR

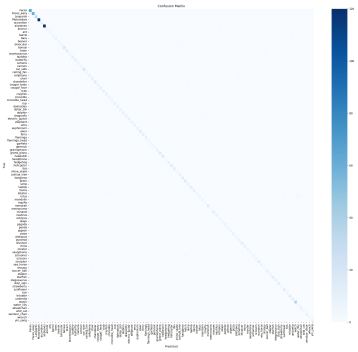
at 64×64 and 128×128).



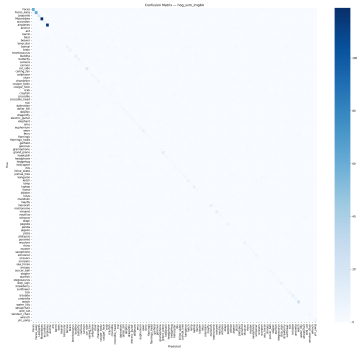
(a) ResNet-18 (128×128, aug, Adam).



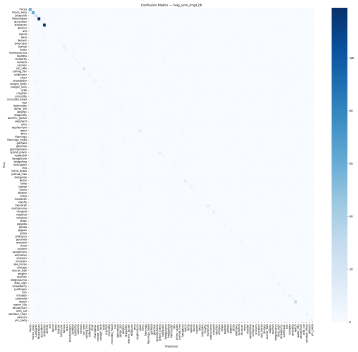
(b) EfficientNet-B0 (128×128, aug, Adam).



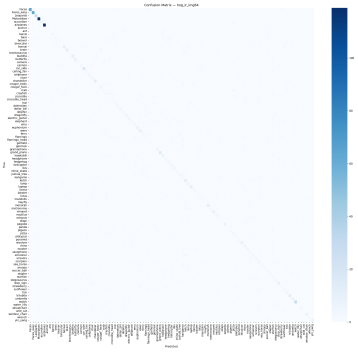
(c) ViT-B/16 (224×224, aug, Adam).



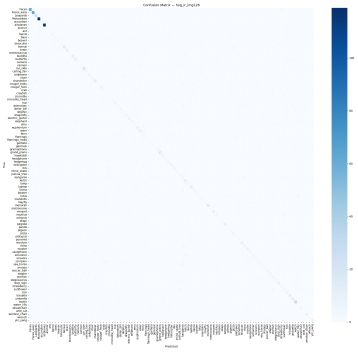
(d) HOG+SVM (64×64).



(e) HOG+SVM (128×128).



(f) HOG+LR (64×64).

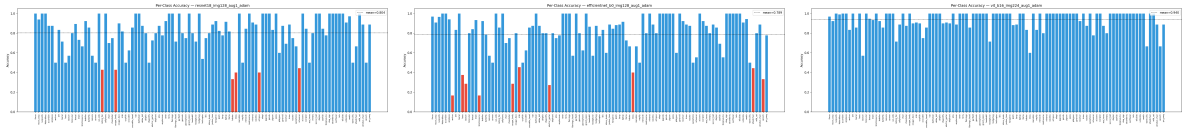


(g) HOG+LR (128×128).

Figure 1: Confusion matrices for all seven baselines: (a)–(c) CNNs (ResNet-18, EfficientNet-B0, ViT-B/16) and (d)–(g) classical HOG+SVM and HOG+LR at 64×64 and 128×128.

4.4 Baseline Per-Class Accuracy

The following figure shows per-class (per-category) accuracy for each CNN baseline: the fraction of test samples correctly classified in that class. Classes are sorted along the horizontal axis; bars indicate how well each of the 101 categories is recognized. We do not include a separate per-class accuracy figure for HOG+SVM or HOG+LR: the classical pipelines are evaluated with the same confusion matrix and classification report (precision, recall, F1 per class). Per-class accuracy and F1 for the HOG baselines can be read from the confusion matrix (diagonal entry divided by row sum for each class) or from the classification report; we do not plot the same bar-chart style as for the CNNs.



(a) ResNet-18 (128×128, aug, Adam). (b) EfficientNet-B0 (128×128, aug, Adam). (c) ViT-B/16 (224×224, aug, Adam).

Figure 2: Per-class accuracy for the three CNN baselines: ResNet-18, EfficientNet-B0, and ViT-B/16.

Summary: confusion matrices and per-class accuracy. The confusion matrices (Figures 1a–1g) show a clear gradient from CNN baselines to the classical baselines: ResNet-18, EfficientNet-B0, and ViT-B/16 exhibit strong diagonal dominance with limited off-diagonal mass, whereas HOG+SVM and HOG+LR have noisier matrices with more diffuse confusions; HOG+LR yields stronger diagonal dominance than HOG+SVM at both resolutions. Within the CNNs, ViT at 224×224 concentrates errors in fewer classes and yields the cleanest diagonal. The per-class accuracy plots (Figures 2a–2c) align with this: all three CNN baselines perform well on many categories, with dips typically in rare or visually ambiguous classes (e.g., fine-grained animals or objects with high intra-class variation). ViT raises the floor across classes compared with ResNet-18 and EfficientNet-B0 at 128×128. Taken together, the confusion matrices and per-class accuracy indicate that (i) learned CNN features yield more separable class representations than HOG, (ii) higher resolution (224) improves both overall accuracy and per-class consistency, and (iii) the remaining errors are concentrated in a small set of difficult categories rather than spread uniformly.

5 Discussion

5.1 Observations and Ablation Studies

Image size: 64×64 vs. 128×128 (and 224). For ResNet-18 and EfficientNet-B0, test accuracy drops sharply at 64×64 (around 60–66% and 54–58% depending on augment/optimizer) compared with 128 (84–86% and 79–85%) and 224 (91–92.5% and 88–94.6%). Larger resolution consistently helps; ViT-B/16 (fixed at 224) reaches the highest accuracy (95.93% without augmentation, 95.55% with). The 64 vs. 128 comparison clearly favors 128 for both architectures.

Data augmentation: with vs. without. Training without augmentation yields similar or slightly better test accuracy at 128 for ResNet (85.79% no aug vs. 85.94% with aug) and EfficientNet (84.64–84.79% no aug vs. 83.95–85.1% with aug); for ViT at 224, no augmentation slightly outperforms (95.93% vs. 95.55%). Overall, the effect of augmentation is modest in this setting, suggesting limited overfitting when augmentation is disabled.

Feature extractor choice: HOG vs. CNN. The classical baselines HOG+SVM and HOG+LR were evaluated at 64×64 and 128×128. HOG+SVM reaches 54.61% test (64) and 49.92% (128); HOG+LR reaches 66.05% (64) and 65.90% (128) test accuracy, with top-5 about 82% in both cases. Thus HOG+LR substantially outperforms HOG+SVM on this dataset. All CNN baselines (ResNet-18, EfficientNet-B0, ViT-B/16) substantially outperform both classical methods. Learned CNN features clearly outperform hand-crafted HOG on Caltech-101 under our setup.

Optimizer: SGD vs. Adam (and AdamW) for CNN. SGD underperforms Adam and AdamW: e.g., ResNet-18 at 128 (83.33% SGD vs. 85.94% Adam/AdamW); EfficientNet-B0 at 128 (80.11% SGD vs. 85.1% AdamW). At 224, ResNet-18 reaches 92.55% (Adam) vs. 91.4%

(SGD); EfficientNet-B0 reaches 94.62% (Adam) vs. 89.55% (SGD). Adam and AdamW are close; Adam is the default and performs well across families.

5.2 Analysis of Performance Drivers

The results indicate that the dominant performance gain comes from moving from hand-crafted HOG features to learned CNN/ViT representations. HOG+LR saturates around 66% accuracy at 64×64, whereas pretrained backbones with a linear head achieve above 90%, showing that learned hierarchical features provide much richer class separation than fixed gradient histograms.

5.3 The Role of Input Resolution and Feature Alignment

Input resolution emerged as a critical bottleneck for convolutional architectures. The sharp decline in accuracy at 64×64 (e.g., 60.60% for ResNet-18) compared to 224×224 (92.55%) indicates that aggressive downsampling discards the fine-grained discriminative textures (such as the specific contours of animal species) that ImageNet-pretrained filters are tuned to detect. Furthermore, because models like ResNet and EfficientNet were originally optimized for 224-pixel inputs, feeding them smaller images likely introduces a distribution shift that hinders the performance of the pre-extracted features. ViT-B/16's superior performance (95.93%) further underscores this, as its patch-based attention mechanism effectively captures global context that global average pooling in CNNs might compress too aggressively at lower resolutions.

5.4 Optimization and Regularization Nuances

Contrary to some common assumptions in deep learning, data augmentation yielded only marginal gains in this head-only transfer learning setup. In several instances, such as with ViT-B/16, training without augmentation slightly outperformed the augmented runs. This phenomenon can be attributed to the fact that we are only training a linear classification head; since the backbone remains frozen, the risk of overfitting is inherently limited, and the noise introduced by augmentation may occasionally interfere with the stable features already learned during pretraining.

Regarding optimization, the consistent advantage of Adam/AdamW over SGD (e.g., a 5% gap for EfficientNet-B0 at 224) suggests that adaptive learning rates are better suited for fine-tuning a single layer. SGD's reliance on a global learning rate appears less efficient at navigating the loss surface of the classification head when the underlying feature extractor is not co-adapting.

5.5 Limitations and Future Trajectories

While the results are robust, several constraints merit acknowledgment. The use of a single stratified split provides point estimates but does not account for the variance inherent in class-imbalanced datasets like Caltech-101. Additionally, the study is limited to linear probing (frozen backbones). A natural extension would be a comparative study of full fine-tuning versus head-only training to determine if the 4% error rate remaining in ViT can be closed by adapting the lower-level filters to Caltech's specific domain.

6 Conclusion

This study evaluated the interplay between architectural design, input resolution, and optimization strategies for image classification. Our findings confirm that representation quality, driven by pretrained transformers and deep residual networks, dominates performance. Code, experiment logs, figures, and the report source are available at `https://github.com/chenxige0509/Caltech101-Mini-Project`.

References

[1] F.-F. Li, M. Andreeto, M. Ranzato, and P. Perona. Caltech-101. Technical Report, Caltech, 2022. `https://data.caltech.edu/records/mzrjq-6wc02`

[2] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

[3] M. Tan and Q. V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *ICML*, 2019.

[4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

[5] N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. In *CVPR*, 2005.

[6] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet large scale visual recognition challenge. *IJCV*, 115(3):211–252, 2015.

[7] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

[8] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[9] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson. How transferable are features in deep neural networks? In *NeurIPS*, 2014.

[10] C. Shorten and T. M. Khoshgoftaar. A survey on image data augmentation for deep learning. *J. Big Data*, 6(1):60, 2019.

[11] C. Cortes and V. Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.

[12] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

[13] S. Marcel and Y. Rodriguez. Torchvision: A library for computer vision in PyTorch. `https://github.com/pytorch/vision`, 2017.

[14] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *J. Mach. Learn. Res.*, 12:2825–2830, 2011.